LAB ASSIGNMENT 3 - COMPUTER NETWORKS
NAME : PARITOSH LAHRE
ID : 12341550

(pcap file link : https://drive.google.com/file/d/1Oz-qeswtuPYivyPTXOJ_KDEWetKMWkVe/view?usp=sharing)

PART 1 : Basic Analysis

1. **How many packets were captured? Plot the graph showing a precise distribution of different protocols captured by Wireshark.**

Details

File

Name:	/home/paritosh/Desktop/IIT_BHILAI/sem6/CSL351-ComputerNetwork/lab/3lab/restart/12341550.pcap
Length:	45 MB
Hash (SHA256):	73b59be3fa373f0c3f44bc7c7f03dec7f7111a7b893f9436ad6b18dbe3e181f5
Hash (SHA1):	5ce395f1269b53b9f6b38df6beb607360b8bea8e
Format:	Wireshark/tcpdump/... - pcap
Encapsulation:	Ethernet
Snapshot length:	262144

Time

First packet:	2026-01-27 21:29:42
Last packet:	2026-01-27 21:40:01
Elapsed:	00:10:19

Capture

Hardware:	Unknown
OS:	Unknown
Application:	Unknown

Interfaces

Interface	Dropped packets	Capture filter	Link type	Packet size limit (snaplen)
Unknown	Unknown	Unknown	Ethernet	262144 bytes

Statistics

Measurement	Captured	Displayed	Marked
Packets	35169	35169 (100.0%)	—
Time span, s	619.307	619.307	—
Average pps	56.8	56.8	—
Average packet size, B	1286	1286	—
Bytes	45219232	45219232 (100.0%)	0
Average bytes/s	73 k	73 k	—
Average bits/s	584 k	584 k	—

Statistics > capture file properties

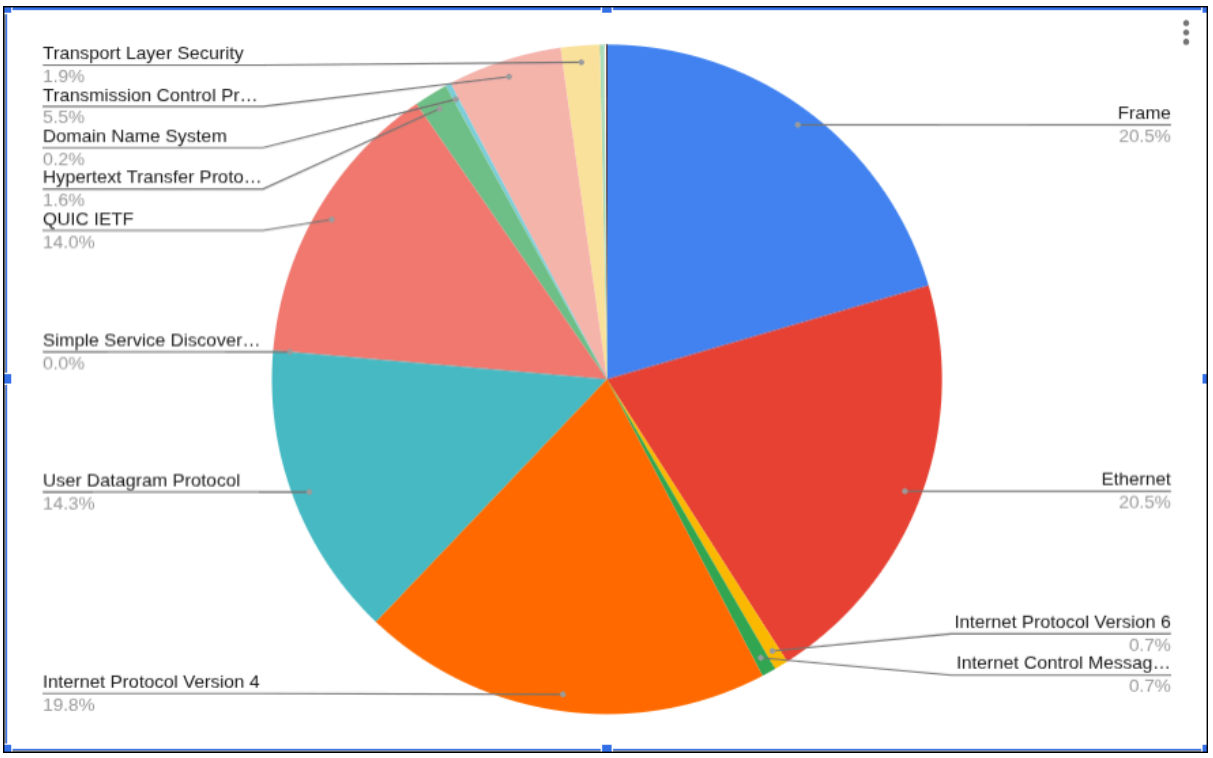
Total packets captured : 35169

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	35169	100.0	45219232	584 k	0	0	0	35169
Ethernet	100.0	35169	1.1	492456	6,361	0	0	0	35169
Internet Protocol Version 6	3.3	1159	0.1	46360	598	0	0	0	1159
Internet Control Message Protocol v6	3.3	1159	0.1	36736	474	1159	36736	474	1159
Internet Protocol Version 4	96.7	34000	1.5	680000	8,784	0	0	0	34000
User Datagram Protocol	69.7	24496	0.4	195968	2,531	0	0	0	24496
Simple Service Discovery Protocol	0.0	8	0.0	1384	17	8	1384	17	8
QUIC IETF	68.4	24039	39.7	17962531	232 k	21236	16564521	213 k	24226
Malformed Packet	0.0	2	0.0	0	0	2	0	0	2
Hypertext Transfer Protocol Version 3	8.0	2801	17.9	8104768	104 k	2801	7628526	98 k	4202
Multicast Domain Name System	0.0	3	0.0	120	1	3	120	1	3
Domain Name System	1.2	424	0.2	69972	903	424	69972	903	424
Data	0.1	22	0.0	704	9	22	704	9	22
Transmission Control Protocol	26.9	9472	56.9	25725432	332 k	6247	6696662	86 k	9472
Transport Layer Security	9.1	3217	59.0	26682112	344 k	2768	21793612	281 k	3709
HyperText Transfer Protocol 2	1.0	355	1.3	610258	7,883	319	554063	7,157	431
Line-based text data	0.0	2	0.0	837	10	2	837	10	2
JavaScript Object Notation	0.1	26	0.1	45478	587	25	17935	231	26
Line-based text data	0.0	1	0.1	27543	355	1	27543	355	1
HTML Form URL Encoded	0.0	4	0.0	1040	13	4	1040	13	4
Hypertext Transfer Protocol	0.3	90	22.9	10351111	133 k	53	39174	506	90
Portable Network Graphics	0.0	4	7.2	3240783	41 k	4	3240783	41 k	4
Media Type	0.0	2	2.4	1084396	14 k	2	1084396	14 k	2
Line-based text data	0.1	20	1.6	721526	9,320	20	721526	9,320	20
JPEG File Interchange Format	0.0	10	11.6	5247918	67 k	10	5247918	67 k	10
HTML Form URL Encoded	0.0	1	0.0	18	0	1	18	0	1
Data	0.0	8	0.0	959	12	8	959	12	8
Hypertext Transfer Protocol	0.0	8	0.0	6172	79	0	0	0	8
Online Certificate Status Protocol	0.0	8	0.0	2216	28	8	2216	28	8
Internet Control Message Protocol	0.1	22	0.1	25800	333	13	25188	325	22
Data	0.0	9	0.0	288	3	9	288	3	9
Data	0.0	10	0.0	14800	191	10	14800	191	10
Address Resolution Protocol	0.0	10	0.0	370	4	10	370	4	10

Statistics > protocol hierarchy

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100	35169	100	45219232	584126.6555	0	0	0	35169
Ethernet	100	35169	1.089041052	492456	6361.379076	0	0	0	35169
Internet Protocol Version 6	3.295515937	1159	0.1025227496	46360	598.8627085	0	0	0	1159
Internet Control Message Protocol v6	3.295515937	1159	0.08123976984	36736	474.5431505	1159	36736	474.5431505	1159
Internet Protocol Version 4	96.67604993	34000	1.503784938	680000	8784.008666	0	0	0	34000
User Datagram Protocol	69.65225056	24496	0.4333731276	195968	2531.447956	0	0	0	24496
Simple Service Discovery Protocol	0.02274730587	8	0.003060644639	1384	17.87804117	8	1384	17.87804117	8
QUIC IETF	68.35281071	24039	39.72321113	17962531	232033.8647	21236	16564521	213974.8471	24226
Malformed Packet	0.005686826466	2	0	0	0	2	0	0	2
Hypertext Transfer Protocol Version 3	7.964400466	2801	17.92327654	8104768	104694.6358	2801	7628526	98542.70367	4202
Multicast Domain Name System	0.0085302397	3	0.0002653738126	120	1.550119176	3	120	1.550119176	3
Domain Name System	1.205607211	424	0.1547394701	69972	903.8744918	424	69972	903.8744918	424
Data	0.06255509113	22	0.001556859701	704	9.094032502	22	704	9.094032502	22
Transmission Control Protocol	26.93281015	9472	56.89046643	25725432	332312.3789	6247	6696662	86505.20153	9472
Transport Layer Security	9.147260371	3217	59.00611492	26682112	344670.4457	2768	21793612	281522.4657	3709
HyperText Transfer Protocol 2	1.009411698	355	1.349554101	610258	7883.105236	319	554063	7157.197344	431
Line-based text data	0.005686826466	2	0.001850982343	837	10.81208126	2	837	10.81208126	2
JavaScript Object Notation	0.07392874406	26	0.1005722521	45478	587.4693325	25	17935	231.6782286	26
Line-based text data	0.002843413233	1	0.06090992434	27543	355.791104	1	27543	355.791104	1
HTML Form URL Encoded	0.01137365293	4	0.002299906376	1040	13.4343662	4	1040	13.4343662	4
Hypertext Transfer Protocol	0.255907191	90	22.89094826	10351111	133712.1305	53	39174	506.0364051	90
Portable Network Graphics	0.01137365293	4	7.166824505	3240783	41863.33229	4	3240783	41863.33229	4
Media Type	0.005686826466	2	2.398085841	1084396	14007.85862	2	1084396	14007.85862	2
Line-based text data	0.05686826466	20	1.595617546	721526	9320.427407	20	721526	9320.427407	20
JPEG File Interchange Format	0.02843413233	10	11.60550007	5247918	67790.8194	10	5247918	67790.8194	10
HTML Form URL Encoded	0.002843413233	1	3.98E-05	18	0.2325178765	1	18	0.2325178765	1
Data	0.02274730587	8	0.002120779053	959	12.38803575	8	959	12.38803575	8
Hypertext Transfer Protocol	0.02274730587	8	0.01364905976	6172	79.72779631	0	0	0	8
Online Certificate Status Protocol	0.02274730587	8	0.00490056974	2216	28.62553412	8	2216	28.62553412	8
Internet Control Message Protocol	0.06255509113	22	0.05705536972	25800	333.2756229	13	25188	325.3700151	22
Data	0.0255907191	9	0.0006368971503	288	3.720286023	9	288	3.720286023	9
Data	0.02843413233	10	0.03272943689	14800	191.1813651	10	14800	191.1813651	10
Address Resolution Protocol	0.02843413233	10	0.000818359223	370	4.779534127	10	370	4.779534127	10

csv data



Pie chart

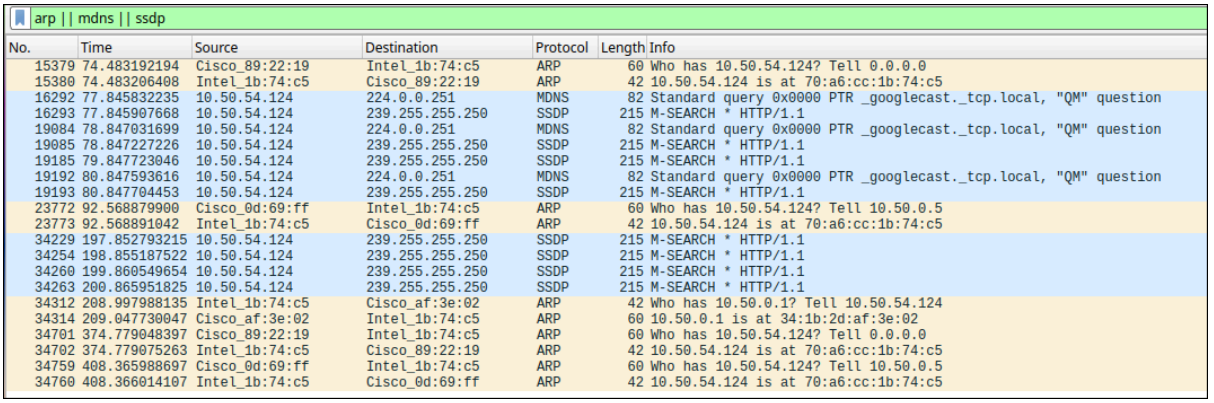
2. **Which is the most common application layer protocol observed in the packet list, and how many times have those protocols been used?**

   The most common application layer protocol observed in the capture is **HTTP**.
   Packet count : **355 times**

3. **Does Wireshark capture packets of the activities performed by the user only? YES or NO? If No, explain the reason with proof.**

   No!! Wireshark captures background system and network noise.

   Search filter : arp || mdns || ssdp



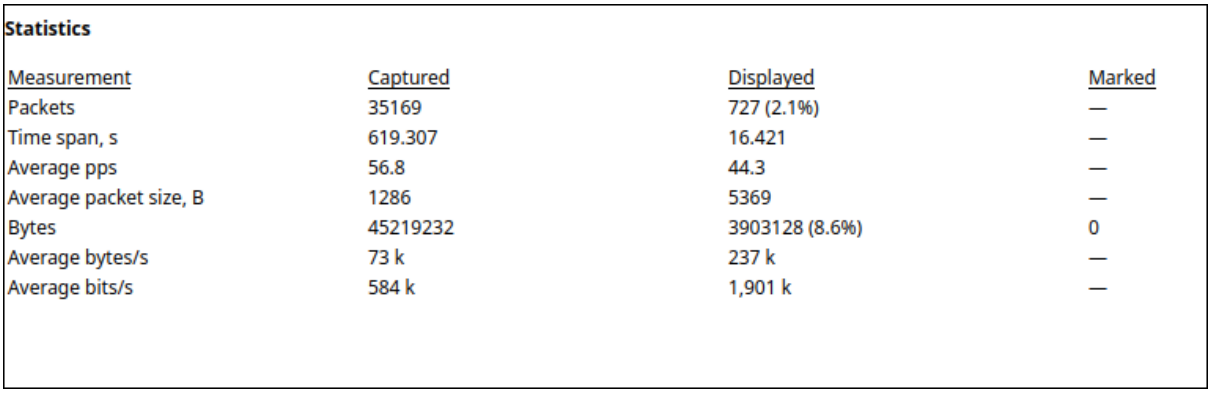
arp || mdns || ssdp

No.	Time	Source	Destination	Protocol	Length	Info
15379	74.483192194	Cisco_89:22:19	Intel_1b:74:c5	ARP	60	Who has 10.50.54.124? Tell 0.0.0.0
15380	74.483206408	Intel_1b:74:c5	Cisco_89:22:19	ARP	42	10.50.54.124 is at 70:a6:cc:1b:74:c5
16292	77.845832235	10.50.54.124	224.0.0.251	MDNS	82	Standard query 0x0000 PTR _googlecast._tcp.local, "QM" question
16293	77.845907668	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
19084	78.847031699	10.50.54.124	224.0.0.251	MDNS	82	Standard query 0x0000 PTR _googlecast._tcp.local, "QM" question
19085	78.847227226	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
19185	79.847723046	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
19192	80.847593616	10.50.54.124	224.0.0.251	MDNS	82	Standard query 0x0000 PTR _googlecast._tcp.local, "QM" question
19193	80.847704453	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
23772	92.568879900	Cisco_0d:69:ff	Intel_1b:74:c5	ARP	60	Who has 10.50.54.124? Tell 10.50.0.5
23773	92.568891042	Intel_1b:74:c5	Cisco_0d:69:ff	ARP	42	10.50.54.124 is at 70:a6:cc:1b:74:c5
34229	197.852793215	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
34254	198.855187522	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
34260	199.860549654	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
34263	200.865951825	10.50.54.124	239.255.255.250	SSDP	215	M-SEARCH * HTTP/1.1
34312	208.997988135	Intel_1b:74:c5	Cisco_af:3e:02	ARP	42	Who has 10.50.0.1? Tell 10.50.54.124
34314	209.047730047	Cisco_af:3e:02	Intel_1b:74:c5	ARP	60	10.50.0.1 is at 34:1b:2d:af:3e:02
34701	374.779048397	Cisco_89:22:19	Intel_1b:74:c5	ARP	60	Who has 10.50.54.124? Tell 0.0.0.0
34702	374.779075263	Intel_1b:74:c5	Cisco_89:22:19	ARP	42	10.50.54.124 is at 70:a6:cc:1b:74:c5
34759	408.365988697	Cisco_0d:69:ff	Intel_1b:74:c5	ARP	60	Who has 10.50.54.124? Tell 10.50.0.5
34760	408.366014107	Intel_1b:74:c5	Cisco_0d:69:ff	ARP	42	10.50.54.124 is at 70:a6:cc:1b:74:c5

Wireshark captures every packet that reaches the network interface, including background system noise, automated network management protocols, and broadcast traffic from other devices on the same local network. These processes happen automatically at the Operating System or Hardware level without any direct user initiation..

4. **Identify and filter the packets received while downloading the Student Handbook. How many data packets were captured?**

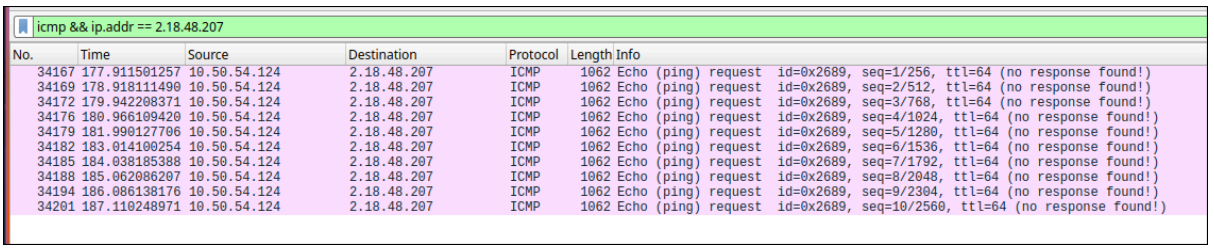
   Packet captured : 727



Statistics

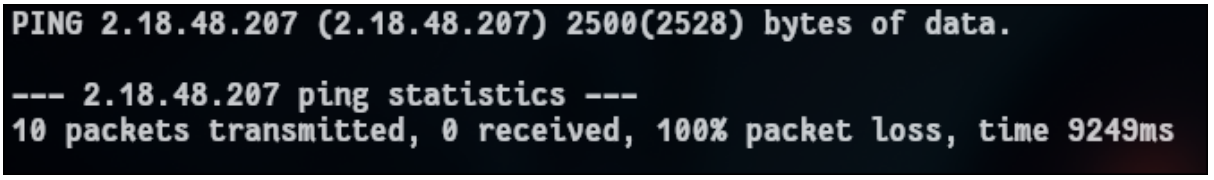
Measurement	Captured	Displayed	Marked
Packets	35169	727 (2.1%)	—
Time span, s	619.307	16.421	—
Average pps	56.8	44.3	—
Average packet size, B	1286	5369	—
Bytes	45219232	3903128 (8.6%)	0
Average bytes/s	73 k	237 k	—
Average bits/s	584 k	1,901 k	—

5. **Can you Identify the packets used to ping tesla.com? If yes, How many packets (including both request and response) per ping request have been captured? Verify the information using the terminal output of the ping command.**



icmp && ip.addr == 2.18.48.207

No.	Time	Source	Destination	Protocol	Length	Info
34167	177.911501257	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=1/256, ttl=64 (no response found!)
34169	178.918111490	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=2/512, ttl=64 (no response found!)
34172	179.942208371	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=3/768, ttl=64 (no response found!)
34176	180.966109420	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=4/1024, ttl=64 (no response found!)
34179	181.990127706	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=5/1280, ttl=64 (no response found!)
34182	183.014100254	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=6/1536, ttl=64 (no response found!)
34185	184.038185388	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=7/1792, ttl=64 (no response found!)
34188	185.062086207	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=8/2048, ttl=64 (no response found!)
34194	186.086138176	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=9/2304, ttl=64 (no response found!)
34201	187.110248971	10.50.54.124	2.18.48.207	ICMP	1062	Echo (ping) request id=0x2689, seq=10/2560, ttl=64 (no response found!)



```
PING 2.18.48.207 (2.18.48.207) 2500(2528) bytes of data.

--- 2.18.48.207 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9249ms
```

By filtering for ICMP traffic to IP 2.18.48.207, I identified the ping attempts to Tesla. Although the terminal reports 10 transmitted packets, Wireshark shows 3 packets per request (a total of 30 packets for the 10 requests) due to IP fragmentation caused by the 2500-byte data size. No response packets were captured, which is verified by the 100% packet loss reported in the terminal statistics.

6. **Save your packet capture as a .pcap file and give the file name as your roll number. Explain why saving captures in .pcap format is useful.**

   Saving captures in the .pcap format is useful because it is the industry-standard "raw" format that preserves the entire binary structure of every packet exactly as it was received by the network interface, including critical timing metadata for each frame. This ensures full compatibility across diverse analysis tools like Wireshark, tcpdump, and Snort, while allowing for retrospective analysis such as recalculating throughput or reapplying different display filters without needing to re-capture live traffic. Unlike exported text or screenshots, a .pcap file maintains the integrity of the data at the link layer, enabling precise troubleshooting of protocol handshakes, latency issues, and retransmissions at a later time or on a different operating system.

7. **Combine filters to display only DNS and HTTP traffic. What filter expression did you use? How many packets match this combined filter?**

Filter expression : dns || http

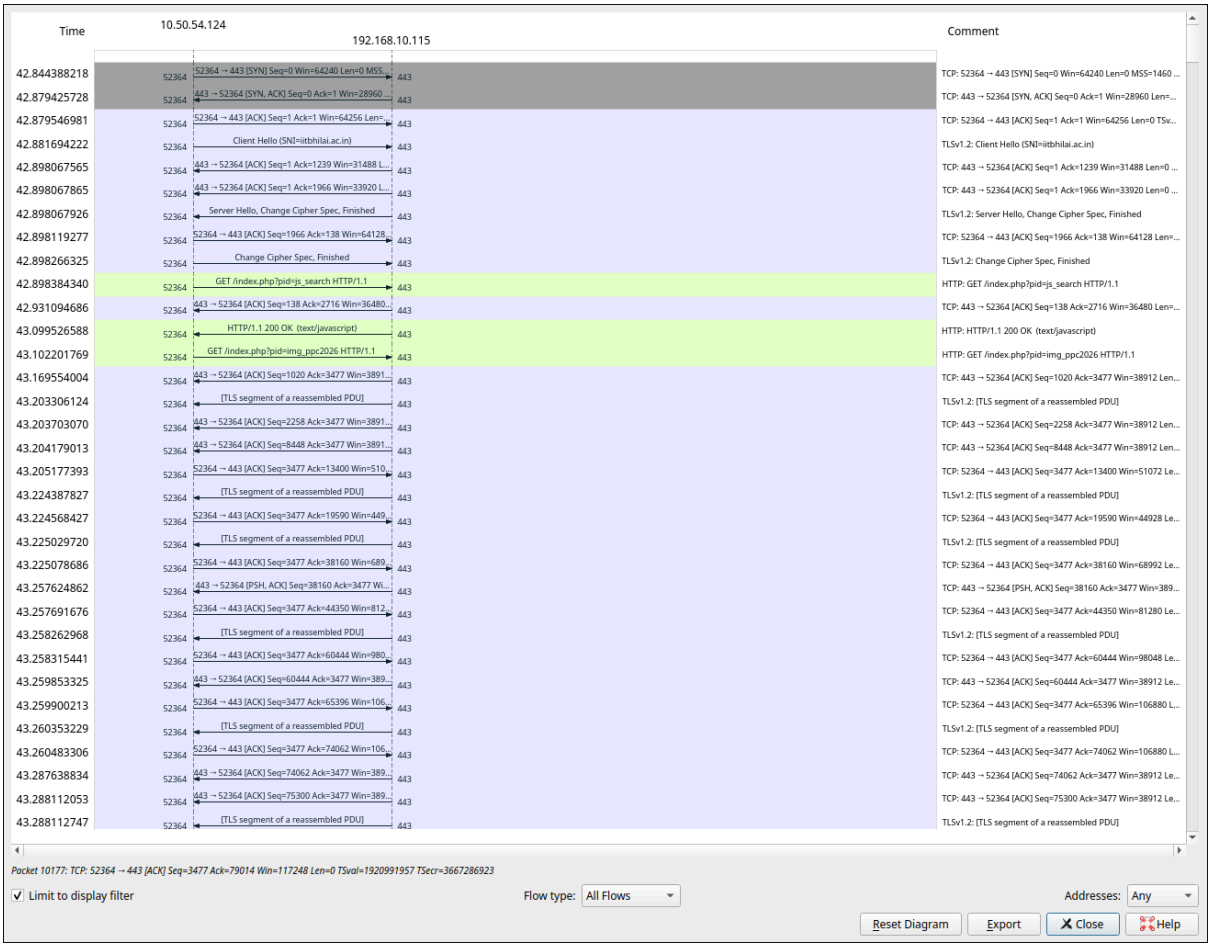
Statistics

Measurement	Captured	Displayed	Marked
Packets	35169	522 (1.5%)	—
Time span, s	619.307	213.406	—
Average pps	56.8	2.4	—
Average packet size, B	1286	754	—
Bytes	45219232	393353 (0.9%)	0
Average bytes/s	73 k	1,843	—
Average bits/s	584 k	14 k	—

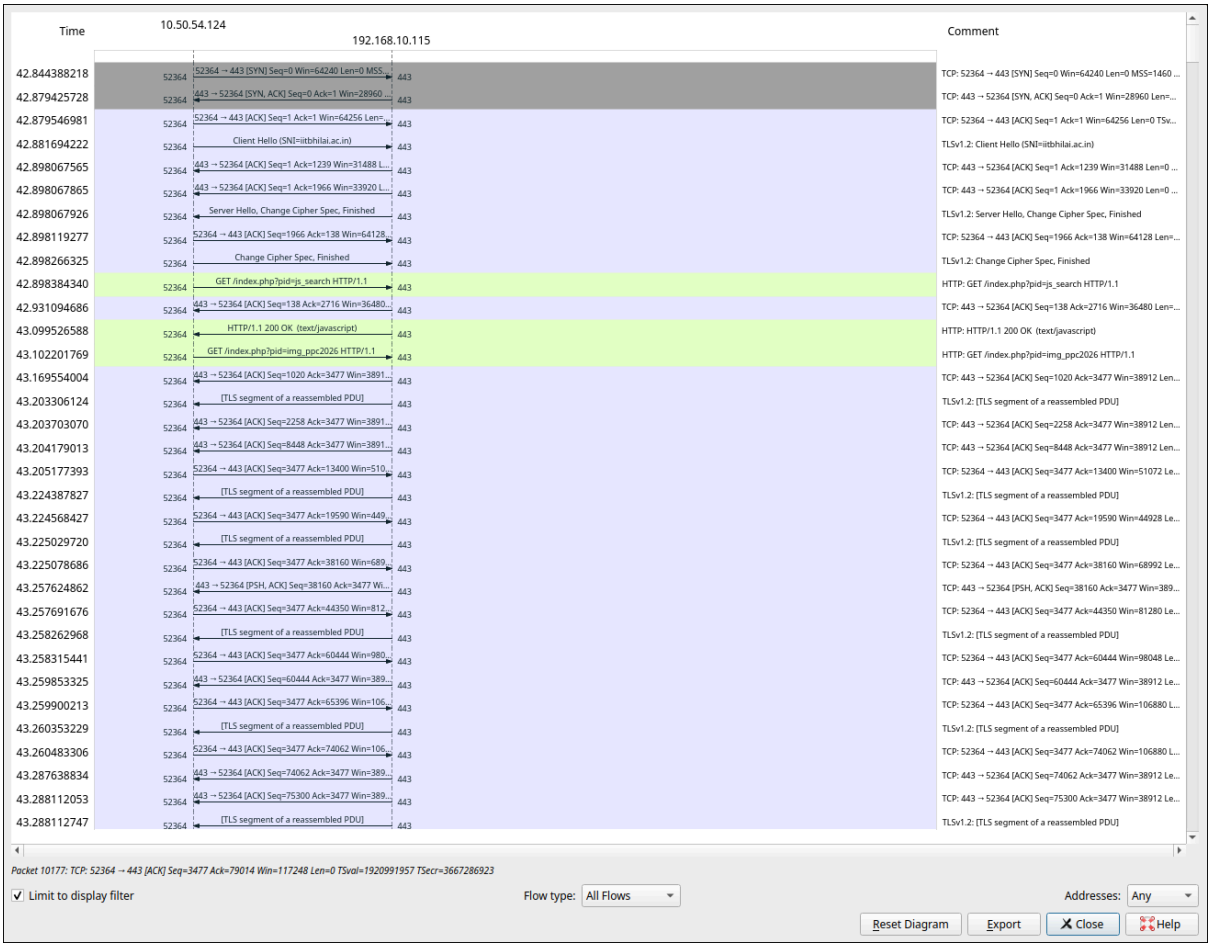
Packets captured : 522

PART 2 : Statistics

1. **Draw the flow graph of packets captured while downloading a Student Handbook file.**



2. **Describe the process of file downloading using the flow graph and packets captured by Wireshark. (Explain on protocol level and mention all the protocols used.)**



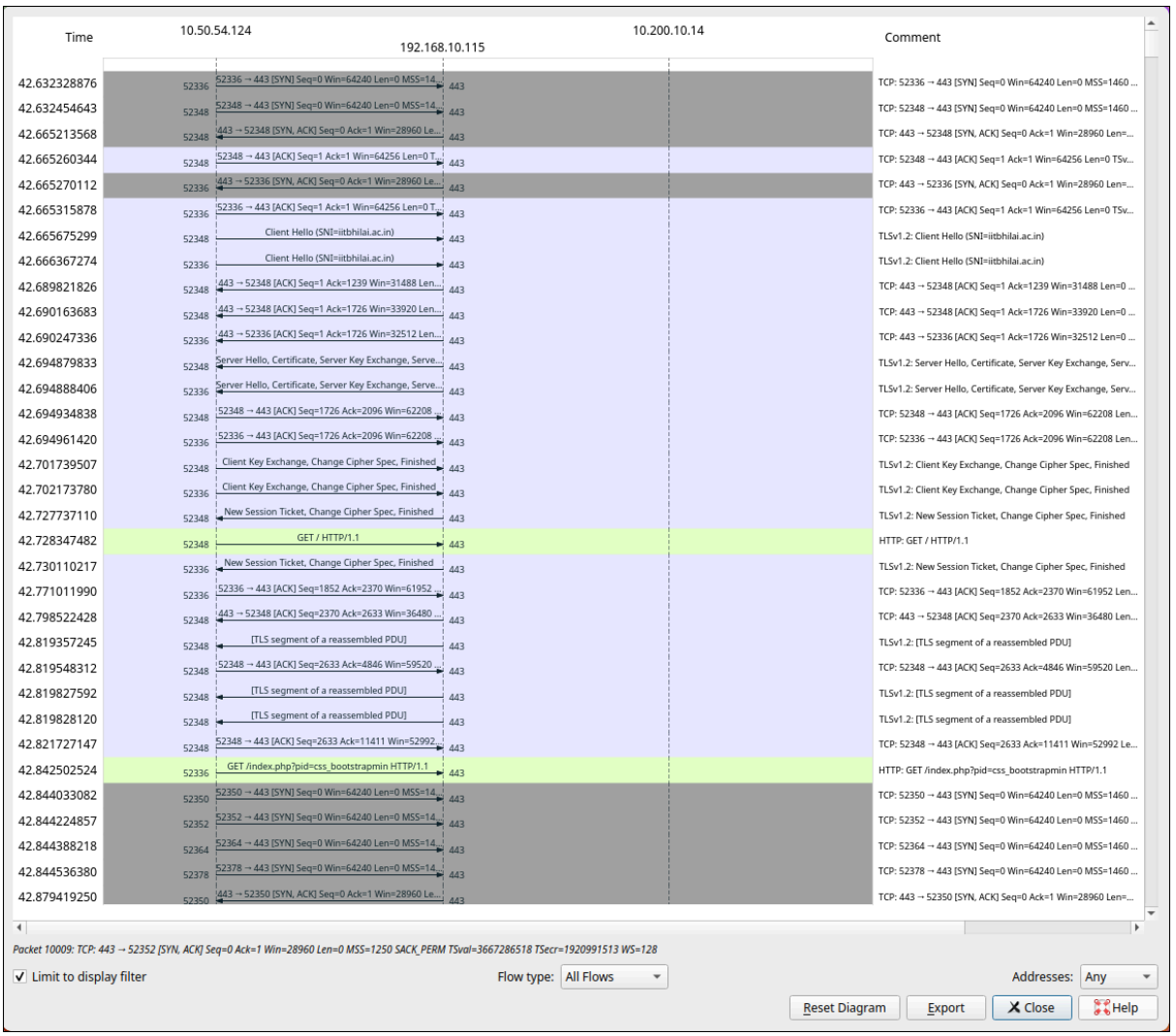
The file download begins with your computer establishing a reliable connection with the institute's server through a TCP Three-Way Handshake, involving the exchange of synchronization and acknowledgment signals to ensure both sides are ready for data transfer. Once this transport channel is live, the nodes initiate a TLS Handshake to negotiate encryption keys, which secures the session and protects the data from being read by unauthorized parties. After the secure tunnel is established, your machine sends an encrypted HTTP request visible in the capture as Application Data specifically asking for the student handbook file. The server then confirms the request and begins streaming the file content in numerous segments, with your computer sending back quiet acknowledgement notes to verify that every piece of the document has arrived correctly. This continuous cycle of data segments and confirmations ensures the entire file is

reassembled perfectly on your side before the connection is eventually closed.

3. **Fill the following table with the data extracted from Wireshark:**

   Total Packets Capture : **35169**
   Total Number of HTTP/HTTPS connections established : **65**
   Average Throughput of streaming videos : **245**
   How many DNS queries were resolved? : **212**
   Average throughput of QUIC Protocols : **245**
   Total Number of SMTP packets captured : **46**
   Average Packet Length : **1286**
   Average Packet Length of TCP Traffic : **2750**
   What is the average time spent on TLS Message Exchange? : **~0.015s - ~0.050s**
   How many packets are retransmitted? : **62**

4. **Draw the flow chart of the traceroute command using Wireshark and explain how the traceroute works.**



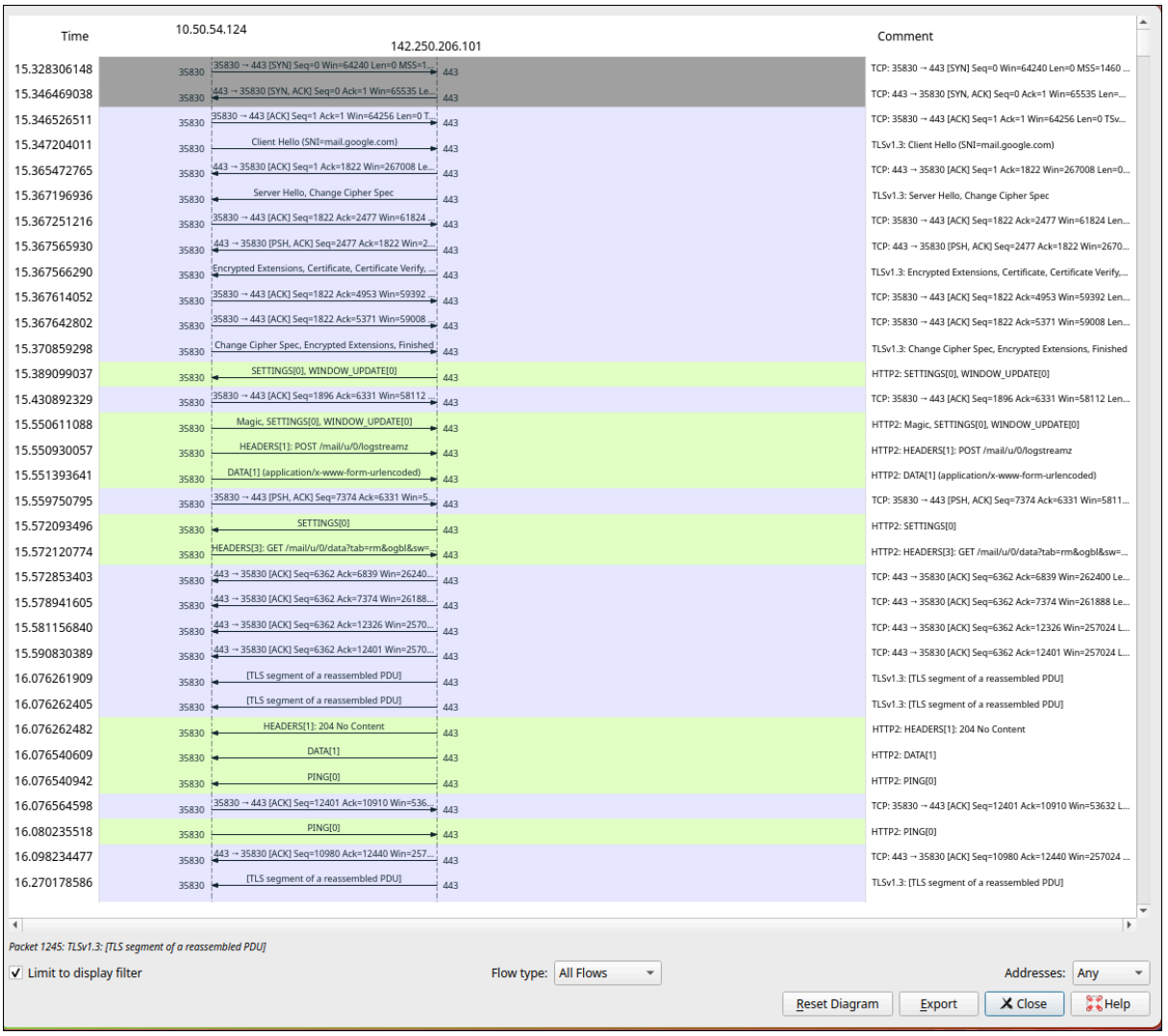

```
traceroute to iitbhilai.ac.in (192.168.10.115), 30 hops max, 60 byte packets
 1  _gateway (10.50.0.1)  1.888 ms  1.822 ms  1.804 ms
 2  10.200.10.14 (10.200.10.14)  1.350 ms  2.067 ms  2.051 ms
 3  192.168.10.115 (192.168.10.115)  2.360 ms  2.345 ms *
```

The traceroute command functions by systematically discovering each router (hop) between our computer and the destination IP 192.168.10.115(iitbhilai.ac.in) through the manipulation of the Time to Live (TTL) field in the IP header. Machine begins by transmitting three UDP probes with a TTL of 1, which causes the first gateway to discard the packets and return ICMP "Time-to-Live Exceeded" messages, effectively revealing its identity. This process repeats with incrementing TTL values (2, 3, 4, etc.), where each subsequent router along the path decrements the TTL until it reaches zero and notifies your computer via ICMP. The trace concludes when the probes finally reach the target destination, which, having no further routers to pass through, responds with an ICMP "Destination Port Unreachable" message. This cycle

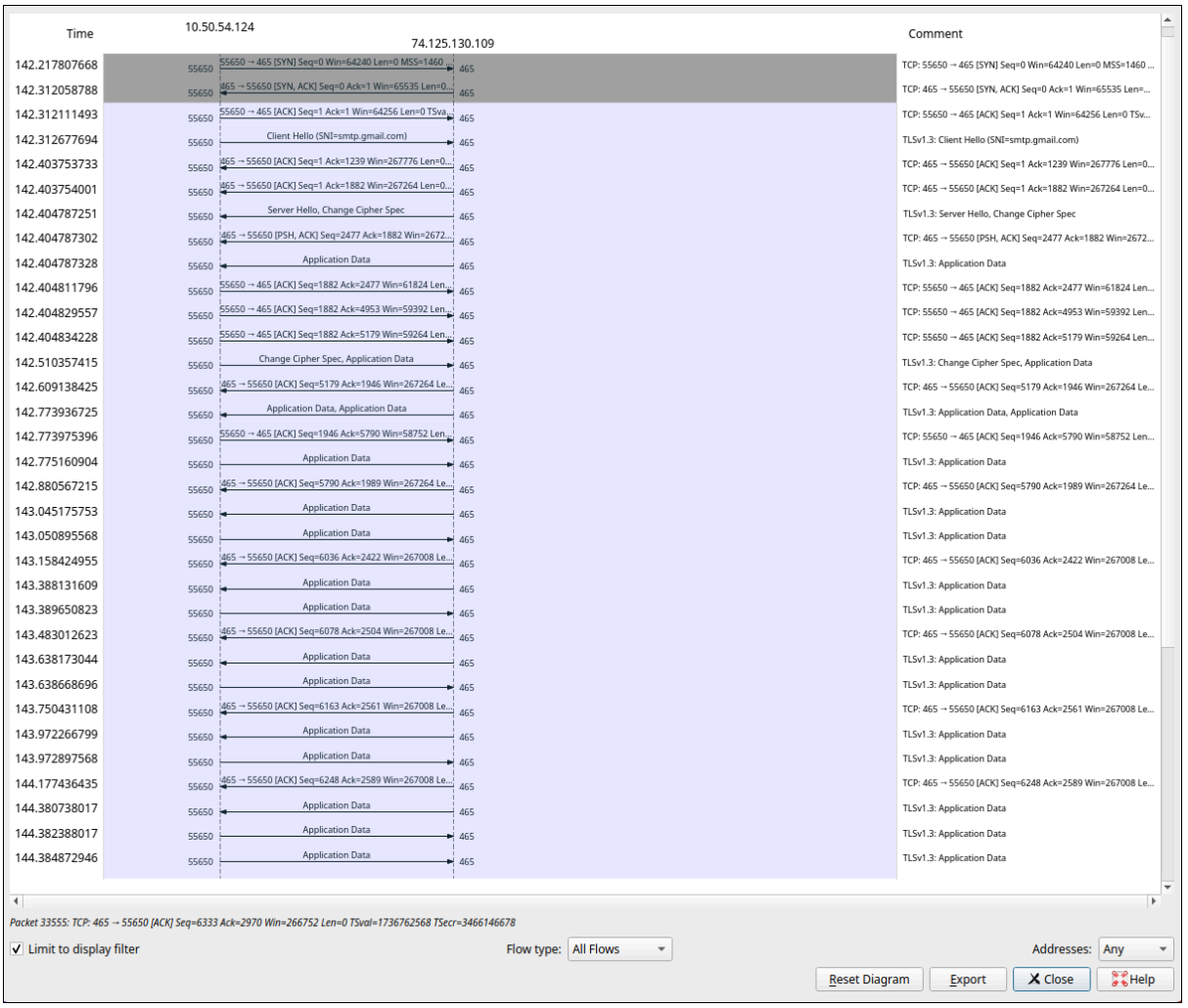
allows Wireshark to capture a complete sequence of intermediate nodes, proving that traceroute is not a single connection but a series of deliberate packet expirations designed to map the network topology.

5. **Can you identify the protocols used for sending emails for both methods? If yes, explain the flow using relevant screenshots/flow graphs. Also, list the protocols used in sequential order.**

Search filter : tls.handshake.extensions_server_name contains "mail.google.com"
Platform : webmail



Search filter : tcp.port == 465
Platform : thundermail

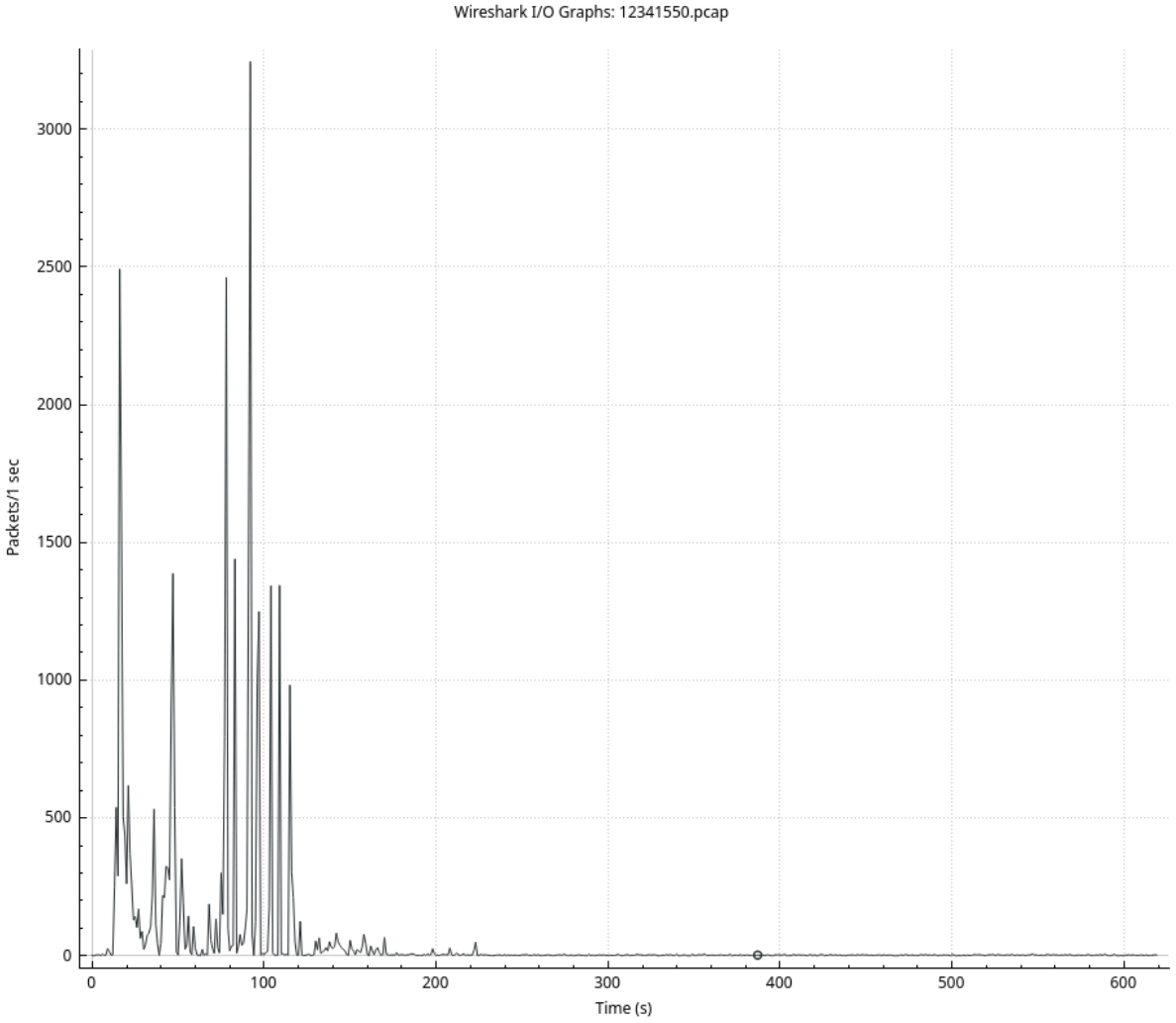


For the webmail platform, packets were filtered using **tls.handshake.extensions_server_name contains "mail.google.com"**, which isolates encrypted TLS connections specifically associated with Gmail. This shows that when email is sent through a browser interface, communication occurs using DNS for server resolution, TCP for connection establishment, TLS for encryption, and HTTPS for data transmission, where the email content is carried inside encrypted HTTP requests. In this case, SMTP is not visible in Wireshark because the browser does not send emails directly using SMTP; instead, SMTP communication occurs internally between mail servers on the backend. For the email client platform (Thunderbird), packets were filtered using **tcp.port == 465**, which corresponds to SMTPS (SMTP over SSL/TLS). This clearly reveals the email transmission process where the client directly communicates with the mail server using DNS -> TCP -> TLS -> SMTP, followed by

standard SMTP command sequences (EHLO, MAIL FROM, RCPT TO, DATA) and server acknowledgments. This demonstrates that web-based email uses HTTPS over TLS, while email clients use direct SMTP over TLS, making SMTP traffic observable only in the client-based method.

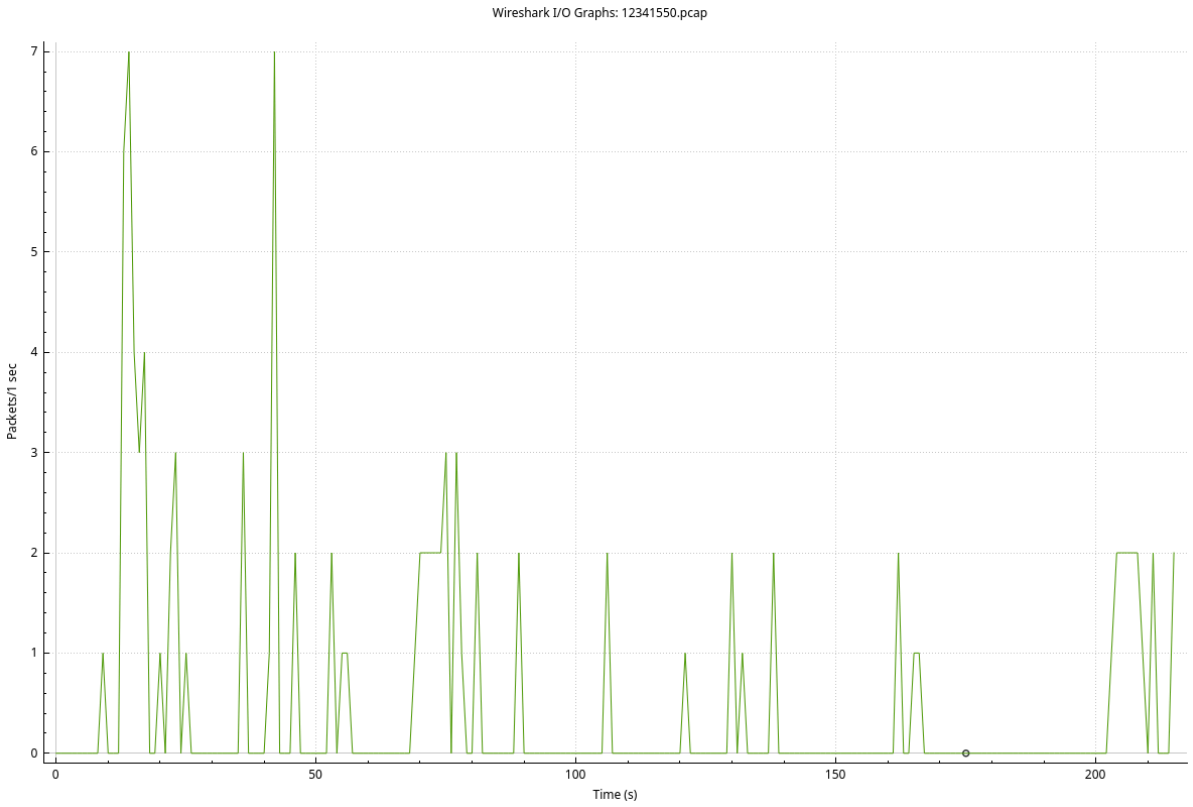
6. Plot the I/O graph for the following:

a. For complete packets captured



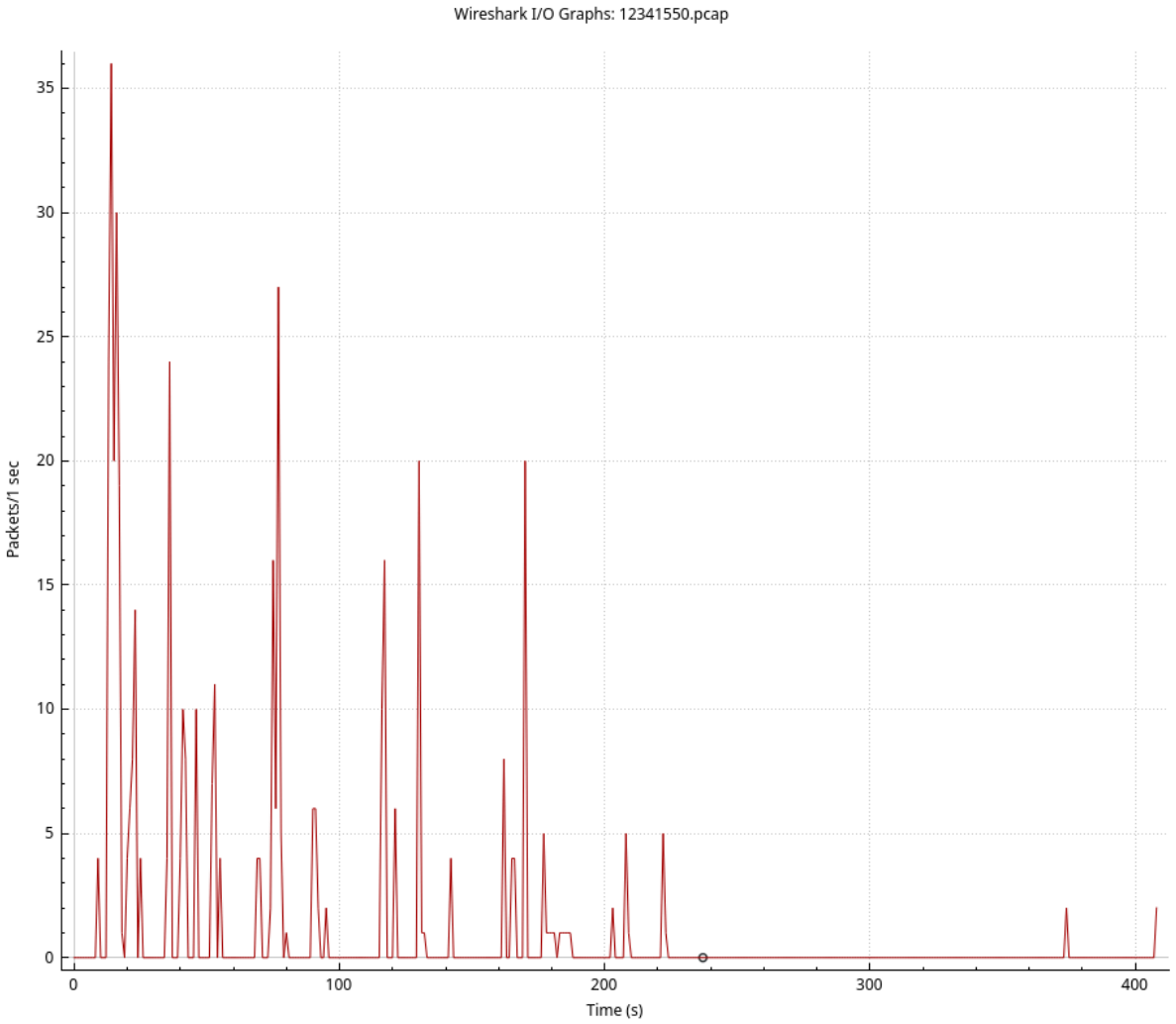
b. For HTTP/HTTPS traffic

Search filter : tcp.flags.syn == 1 && tcp.flags.ack == 0 && (tcp.port == 80 || tcp.port == 443)



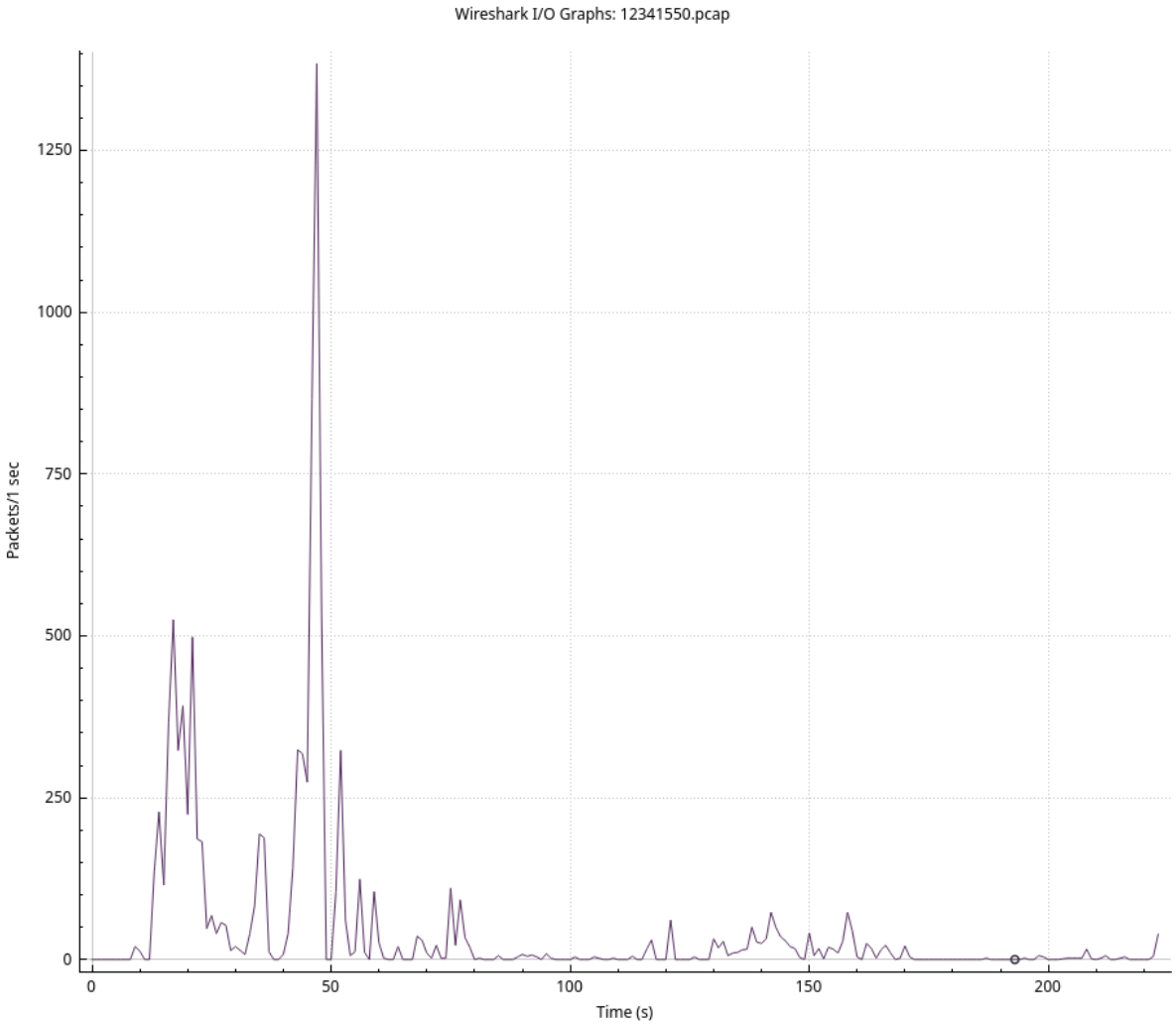
c. For All the ARP, DNS/MDNS, and ICMP Queries

Search filter : arp || mdns || dns || icmp



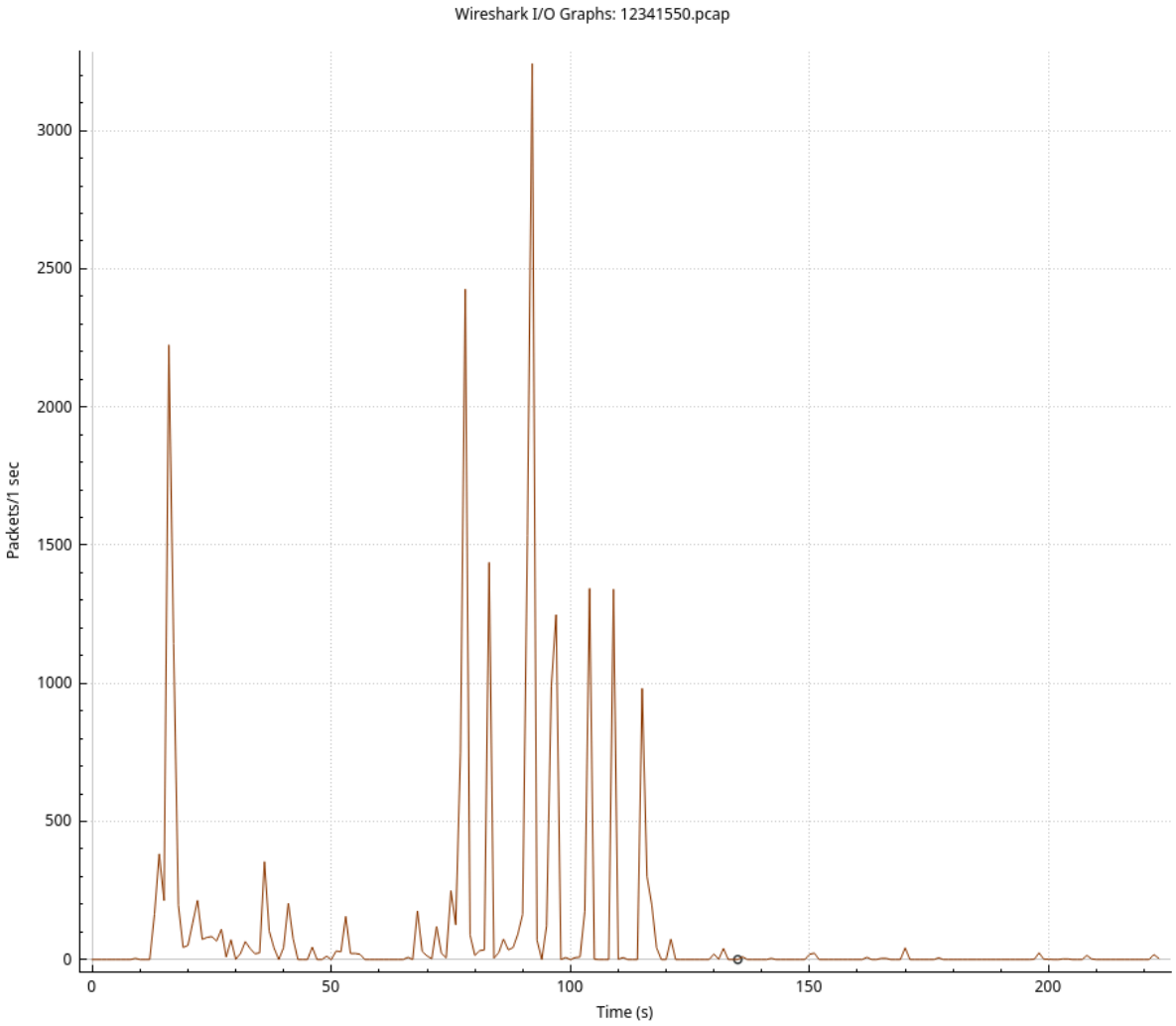
d. For TCP/TLS Protocols

Search filter : tcp || tls



e. For UDP protocol

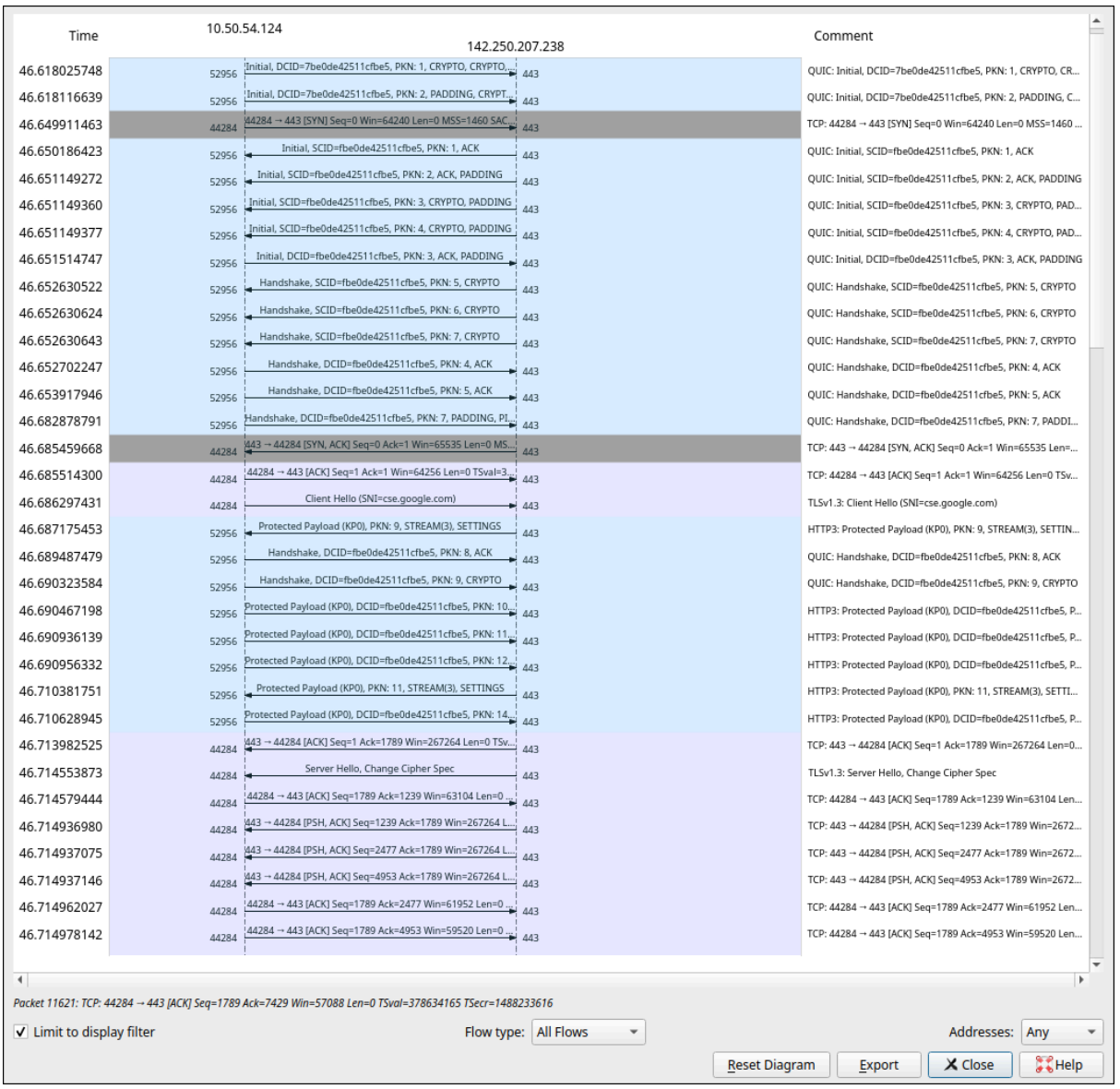
Search filter : udp



7. Plot Flow Graphs for the following:

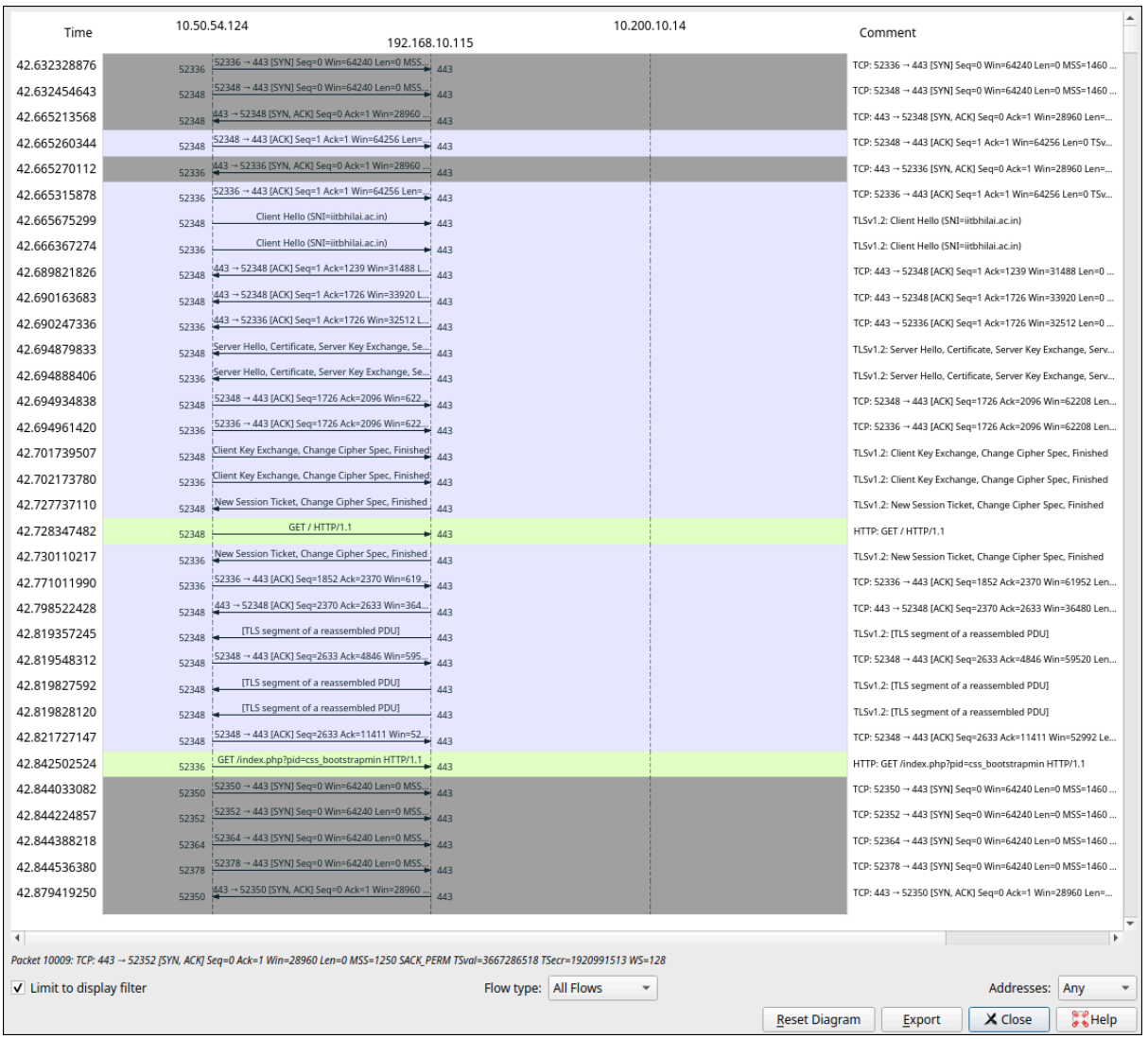
a. For Youtube.com

Search filter : ip.addr == 142.250.207.238



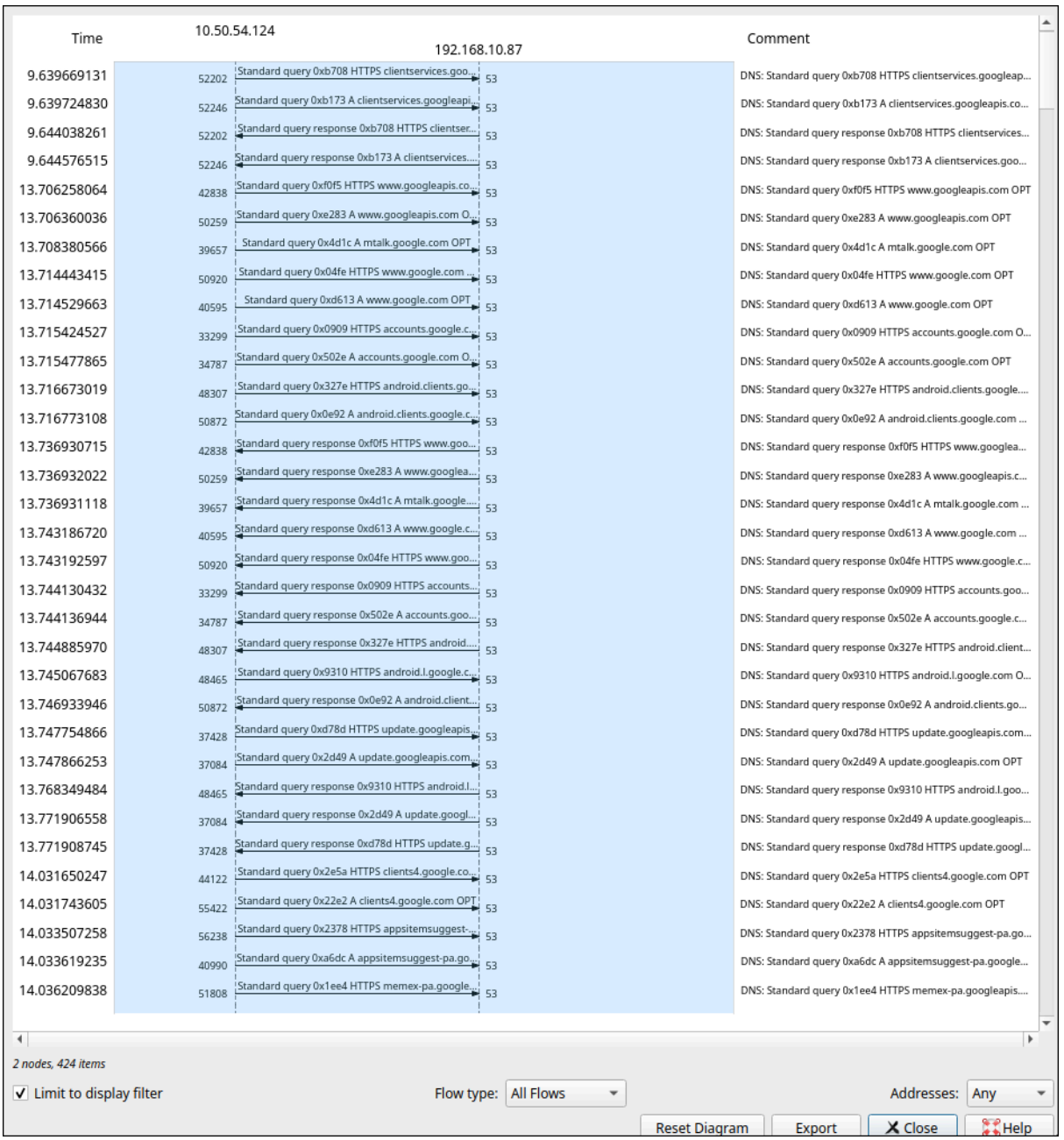
b. For Accessing iitbhilai.ac.in

Search filter : ip.addr == 192.168.10.115



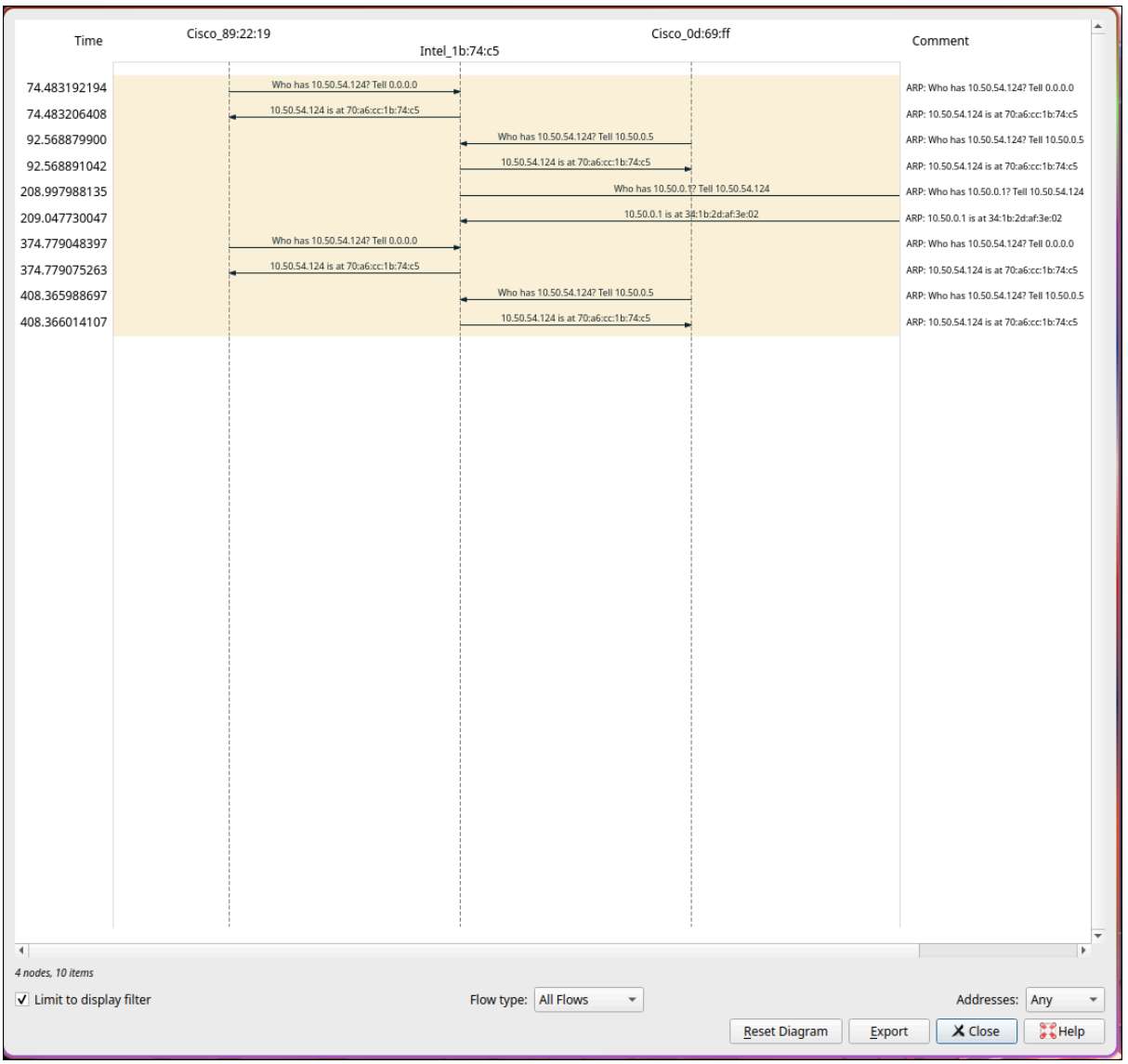
c. DNS Query

Search filter : dns



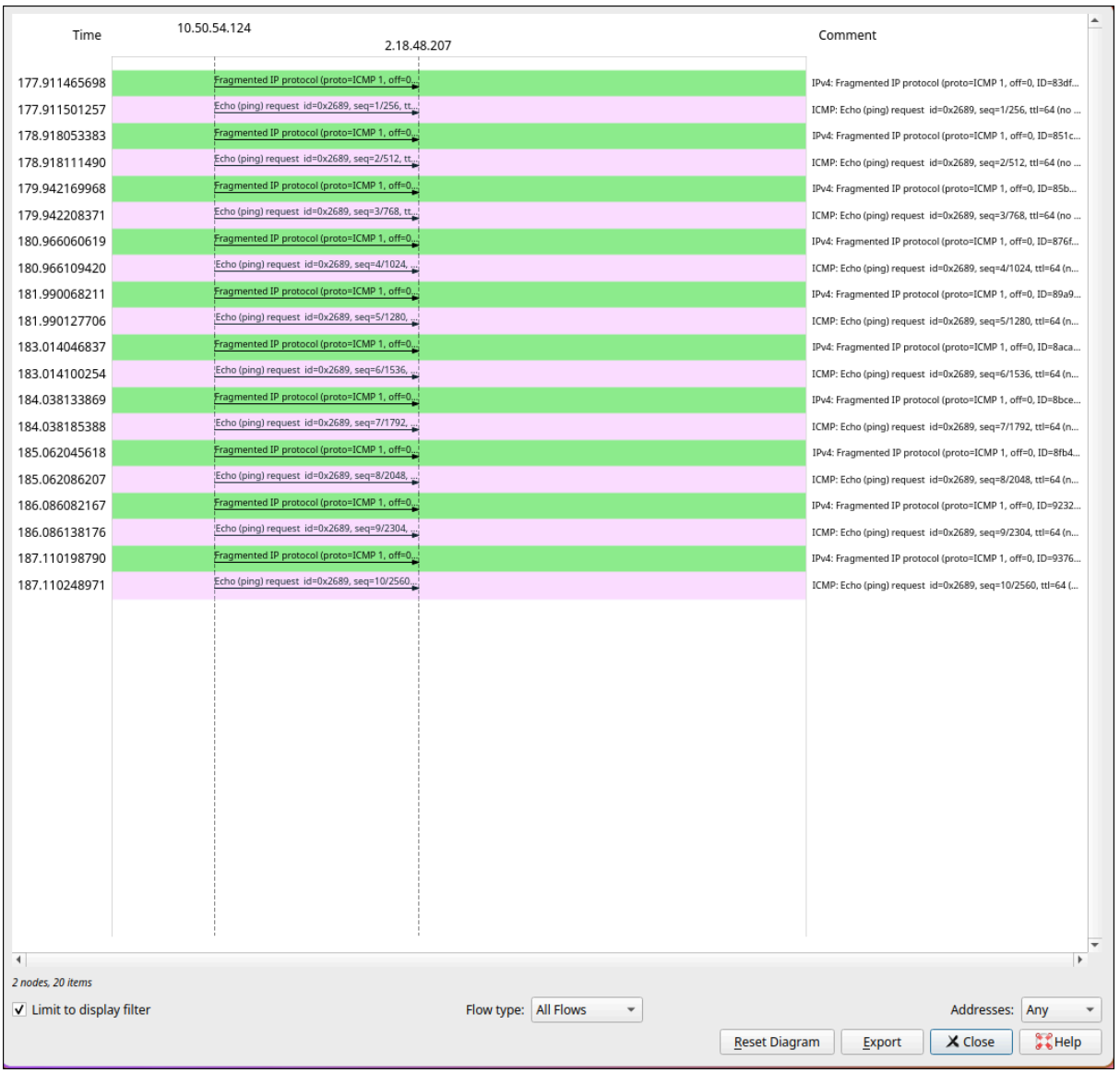
d. ARP Query

Search filter : arp



e. Pinging tesla.com

Search filter : ip.addr == 2.18.48.207


Time
10.50.54.124
2.18.48.207
Comment
177.911465698 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=83df...
177.911501257 Echo (ping) request id=0x2689, seq=1/256, tt... ICMP: Echo (ping) request id=0x2689, seq=1/256, ttl=64 (no ...
178.918053383 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=851c...
178.918111490 Echo (ping) request id=0x2689, seq=2/512, tt... ICMP: Echo (ping) request id=0x2689, seq=2/512, ttl=64 (no ...
179.942169968 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=85b...
179.942208371 Echo (ping) request id=0x2689, seq=3/768, tt... ICMP: Echo (ping) request id=0x2689, seq=3/768, ttl=64 (no ...
180.966060619 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=876f...
180.966109420 Echo (ping) request id=0x2689, seq=4/1024, ... ICMP: Echo (ping) request id=0x2689, seq=4/1024, ttl=64 (n...
181.990068211 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=89a9...
181.990127706 Echo (ping) request id=0x2689, seq=5/1280, ... ICMP: Echo (ping) request id=0x2689, seq=5/1280, ttl=64 (n...
183.014046837 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=8aca...
183.014100254 Echo (ping) request id=0x2689, seq=6/1536, ... ICMP: Echo (ping) request id=0x2689, seq=6/1536, ttl=64 (n...
184.038133869 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=8bce...
184.038185388 Echo (ping) request id=0x2689, seq=7/1792, ... ICMP: Echo (ping) request id=0x2689, seq=7/1792, ttl=64 (n...
185.062045618 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=8fb4...
185.062086207 Echo (ping) request id=0x2689, seq=8/2048, ... ICMP: Echo (ping) request id=0x2689, seq=8/2048, ttl=64 (n...
186.086082167 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=9232...
186.086138176 Echo (ping) request id=0x2689, seq=9/2304, ... ICMP: Echo (ping) request id=0x2689, seq=9/2304, ttl=64 (n...
187.110198790 Fragmented IP protocol (proto=ICMP 1, off=0,... IPv4: Fragmented IP protocol (proto=ICMP 1, off=0, ID=9376...
187.110248971 Echo (ping) request id=0x2689, seq=10/2560, ... ICMP: Echo (ping) request id=0x2689, seq=10/2560, ttl=64 (...
2 nodes, 20 items
Limit to display filter
Flow type: All Flows
Addresses: Any
Reset Diagram
Export
Close
Help